CloudSage — Project Index and Engineering Decisions

Single reference for project rules, scope and decisions

1. Runtime AI Decision

Exactly one LLM API/provider is used by the working product. Multiple AI tools can be used during development, but they are not runtime dependencies.

2. First Service

EC2 is the first AWS service/resource family. Do not add more services until the EC2 lifecycle is stable.

3. First Optimization

EC2 rightsizing/underutilization analysis is the first optimization problem.

4. Infrastructure Change Mechanism

Terraform is the preferred mechanism for infrastructure changes.

5. Safety Decision

The AI never gets unrestricted AWS write permissions. Human approval and deterministic policy gates protect infrastructure changes.

6. Financial Decision

Financial calculations are deterministic. The LLM may explain savings but does not decide the numeric result.

7. Architecture Decision

Use a plugin architecture for AWS services so additional optimizers can be added without rewriting the core platform.

8. MVP Decision

Start with a CLI/API workflow before building a sophisticated UI or Kubernetes deployment.

9. Documentation Map

- 01_project_description.docx — purpose, scope, problem and success criteria
- 02_architecture.docx — architecture and data flow
- 03_build_from_first_command.docx — complete development runbook
- 04_project_versions.docx — V0 to V10 progression
- 05_problems_and_risks.docx — expected challenges and failure modes
- 06_aftercare_operations.docx — maintenance and post-change operation
- 07_ai_design.docx — runtime AI design and development/runtime separation
- 08_apis_data_contracts.docx — APIs, AWS integrations and database contracts
- 09_security_testing.docx — security, tests and quality gates
- 10_roadmap_and_future.docx — future expansion
- 11_project_index_and_decisions.docx — project rules and decisions

10. Core Loop

DISCOVER → MEASURE → DETECT → REASON → RECOMMEND → APPROVE → PLAN → APPLY → VERIFY → AUDIT

11. First Milestone

A single EC2 instance can be discovered, its cost and utilization collected, an optimization candidate detected, a single runtime LLM can produce a structured recommendation, and the recommendation can be validated and stored without changing infrastructure.